B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

# Dungeons Together
# An AI enhanced D&D application

**By**
**Eoin Ocathasaigh**
**G00417466**

May 3, 2026

## Minor Dissertation

**Department of Computer Science & Applied Physics,**
**School of Science & Computing,**
**Atlantic Technological University (ATU), Galway.**

# Contents


**1 Introduction** **2**
1.1 Background and Motivation . . . . . 2
1.2 Objectives . . . . . 3
1.2.1 Overall Context Objectives . . . . . 3
1.2.2 AI Objectives . . . . . 3
1.2.3 Content Overview . . . . . 4

**2 Methodology** **6**
2.1 Introduction . . . . . 6
2.2 Software Development Methodology . . . . . 6
2.2.1 Tools and Technologies . . . . . 7
2.2.2 Software Design Approach . . . . . 7
2.3 Research Methodology . . . . . 8
2.3.1 Literature Review Methodologies . . . . . 9
2.3.2 Experimentation . . . . . 9
2.4 Summary . . . . . 10

**3 Technology Review** **11**
3.1 Literature Review - Understanding Modern Adaptive Algorithms . . 11
3.1.1 Understanding AI in Games . . . . . 14
3.1.2 Applications of LLM's in Games . . . . . 15
3.2 Research into Development Environments . . . . . 16
3.3 Application of advanced technologies in Unity . . . . . 16
3.3.1 Application of AI in Unity . . . . . 18
3.3.2 Unity Netcode and Networking . . . . . 19
3.3.3 Unity Networking Tools . . . . . 20
3.4 Research Gaps . . . . . 20

**4 System Design** **23**
4.1 Overall System Architecture . . . . . 23
4.2 System Interaction and State Management . . . . . 24

4.3 Core Components & Implementation . . . . . 26
4.3.1 Character & Campaign Management . . . . . 26
4.3.2 Character Management . . . . . 26
4.3.3 Campaign Management . . . . . 30
4.4 AI Architecture . . . . . 37
4.4.1 Selection, Training & Installation . . . . . 40
4.4.2 AI Decision Pipeline . . . . . 42
4.4.3 Prompt Design . . . . . 43
4.4.4 Integration with Combat Systems . . . . . 44
4.4.5 Model Constraints . . . . . 45
4.5 Networking Architecture . . . . . 47
4.6 Constraints and Design Considerations . . . . . 48
4.6.1 Engine Selection & Game Design . . . . . 48
4.7 Test Cases & Validation . . . . . 49
4.8 Scalability and Future Improvements . . . . . 49

**5 System Evaluation** **51**
5.1 Evaluation of Project Objectives . . . . . 51
5.2 AI Evaluation . . . . . 52
5.2.1 Advantages & Disadvantages . . . . . 53

**6 Conclusion** **57**
6.1 Summary . . . . . 57
6.2 Key Achievements . . . . . 58
6.2.1 Deepened Understanding of AI systems . . . . . 58
6.2.2 AI Design & Training Process . . . . . 58
6.2.3 Software Design Principle Application . . . . . 59
6.3 Limitations Encountered . . . . . 59
6.4 Final Remarks . . . . . 59

**A App Repo & Links** **62**

# List of Figures


3.1 A comparison of 5 of the most popular engines for games development as recent as 2025 . . . 17

4.1 Overall System Architecture . . . 25
4.2 Outlines the CharacterData.cs structure used during character creation . . . 28
4.3 Overview of the Character Creation method used by the CharacterManager.cs script . . . 29
4.4 An example of a serialized character file as stored on a player's local device . . . 30
4.5 An overview of the basic flow of actions for campaign and character interaction . . . 31
4.6 An overview of the campaign data class used to outline it's elements 32
4.7 Similarly highlights the scene data class used to outline the elements of a scene . . . 32
4.8 An example of a serialized character file as stored on a player's local device . . . 33
4.9 An example of a serialized character file as stored on a player's local device . . . 34
4.10 An overview of the player synchronization methods used to ensure consistency of state during game-play . . . 38
4.11 An Overview of the AI architecture . . . 39
4.12 An example of one of the methods used within the training data generation, used to randomly generate dice values to ensure variation in data. . . . 40
4.13 Provides an overview of the cell used within colab to load the desired model directly from HuggingFace. . . . 41
4.14 Provides an overview of the initial system prompt which is sent directly to the DM brain model. . . . 43
4.15 Displays the construction of the observation prompt within the system. . . . 44

4.16 Displays the temperature parameter as configured within the DM-Brain GameObject. . . . . 44
4.17 Highlights the training loss documented while training the Qwen2.5-0.5 model . . . . . 47
4.18 Displays an example of the testing and validation documentation kept throughout the development process . . . . . 49

# List of Tables

1.1 Google Trends Interest Over Time: Dungeons & Dragons vs Warhammer (Worldwide, Jan 2010 – Mar 2026) . . . . . 4

3.1 Applications of LLMs in Modern Game Development . . . . . 21
3.2 Comparison of Game Engines . . . . . 22

4.1 Overview of Core Management Scripts . . . . . 35
4.2 Comparison of Models during Design Phase . . . . . 46

5.1 Overall Project Objectives evaluation . . . . . 55
5.2 Evaluation of the AI Objectives established . . . . . 56

# Chapter 1

# Introduction

Tabletop role playing games have experienced a steady increase in players and popularity over the last 20 years as highlighted in table 1.1, especially in the early 2020's thanks to the increased attention to online game play podcasts such as *Dimension20*, *Critical Role* and *Legends of Avantris*. Some of the most notable games include **Warhammer 40,000**, **Pathfinder**, **Mythic Bastionland**, **Call of Cthulu**, and **The Dark Eye**. However, the undisputed most popular of these is undoubtedly the 1974 game of **Dungeons and Dragons (D&D)** which is the main focus of this project.

These aforementioned games are what is known as "Tabletop Roleplaying Games" (TTRPG's), meaning their main form of game play is based on an in person format, where players are physically interacting with one another and the items involved with the games such as dice or boards. There does exist some digital adaptation of their systems such as the most recent **Warhammer 40,000: Space Marine 2** and **Baldur's Gate 3**. Critically acclaimed applications like these are one of the biggest inspirations behind the project, focusing mainly on the area of building interactive applications that adapt the behavior of Dungeons and Dragons .

## 1.1 Background and Motivation

Dungeons & Dragons is a collaborative tabletop role-playing game with a long history of inspiring creativity, storytelling, and shared experience. Its adaptability as a system that encompasses character progression, tactical combat, and narrative-driven decision making has made it the foundation for some of the most ambitious titles in modern game development. Games such as Baldur's Gate 3 demonstrate how the core mechanics of D&D can be translated into rich, interactive digital experiences that remain faithful to the source material while offering both solo

and cooperative play. It is this intersection of a well-established game system and modern software capability that motivates the development of Dungeons Together.

The project is also inspired by a broader academic context. Over the course of this degree, significant work has been invested in areas such as database management, networking, and software engineering. Dungeons Together represents an opportunity to apply that accumulated technical knowledge within a domain that demands extensive knowledge and creativity, while simultaneously extending into new territory, specifically integrating advanced **AI** and **Large Language Model (LLM)** technologies into a mechanism for enhancing player experience. These AI driven systems as well as the networking functionalities are discussed in greater detail in chapter 3.

## 1.2 Objectives

The main objectives of this project can be divided into two main areas, the broad objectives applying to the project as a whole, as well as the objectives focusing on the AI functionalities and research relating to this project.

### 1.2.1 Overall Context Objectives

The overall objectives of the application are as follows:

- **Application of Project Management techniques** - Apply many of the project management techniques acquired to enhance the development process, such as user stories, models for management, etc.
- **Application of Software Design Principles and best practices**
- **Effective Multiplayer functionalities** - session creation & state management during connections and game-play
- Implement AI learning functionalities - To build an effective AI driven gameplay mechanic to enrich the players experience
- **Adapting the core systems of Dungeons & Dragons** - Campaign & character creation, scene creation, and management.

### 1.2.2 AI Objectives

The overall objectives relating to the AI functionalities are as follows:

- Investigate the application of local LLM's within a real-time game context

- Research the history of "AI" within video games
- Evaluate suitable AI models and data sets for training and integration
- Design and train a specific model for player behavior classification
- Implement an "Adaptive" AI within Unity to enhance the user experience
- Implement an AI driven assistant to help player's during onine sessions
- Ensure AI functionalities are Isolated and not reliant on external services or web calls

| **Period** | **DnD Interest (0–100)** | **Warhammer Interest (0–100)** |
|---|---|---|
| 2010 | ∼40 | ∼22 |
| 2011 | ∼48 | ∼20 |
| 2013 | ∼25 | ∼20 |
| 2015 | ∼20 | ∼25 |
| 2017 | ∼22 | ∼30 |
| 2019 | ∼25 | ∼40 |
| 2021 | ∼35 | ∼55 |
| 2022 | ∼38 | ∼65 |
| 2023 | ∼45 | ∼75 |
| 2024 (Peak) | ∼100 | ∼100 |
| 2025 | ∼50 | ∼70 |
| 2026 (Mar) | ∼55 | ∼95 |

Table 1.1: Google Trends Interest Over Time: Dungeons & Dragons vs Warhammer (Worldwide, Jan 2010 – Mar 2026)

### 1.2.3 Content Overview

The main content of this dissertation is as follows. Section 1 serves as a basic introduction to the general basis for this project, the background, and an explanation of some of the basic concepts that will be covered throughout the project. Section 2 covers the methodology aspects of the project, specifying and describing the main approaches to solving the problem from both a software development standpoint as well as a research based one and explaining our reasoning behind them. Section 3 explores the main technology review for the application, highlighting important aspects such as understanding AI and advanced learning algorithms within systems like recommendation systems and the previous AI systems applied

to video games. Section 4 will highlight the system design and describe things such as overall architecture, design decisions, technological specifications, and restrictions, as well as providing graphs and diagrams of the system itself. Section 5 will serve as an evaluation of our system, where a comparison of the goals set out in the introduction will be compared to a critical evaluation of the system after completion. Finally, Section 6 will summarize the context and objectives of the project whilst highlighting the results and findings of the overall evaluation.

### External Links

Below are some useful links related to the project for ease of access to those who are interested

- To see the complete code base and to get a better understanding of it's structure and cloning, please visit the GitHub Repository here
- Similarly, you can access the fully published game on our Itch.io page

# Chapter 2

# Methodology

## 2.1 Introduction

This chapter covers the main methodologies used throughout this project. Focusing on areas such as the approach to solving the problems wished to solve, the development methodologies used, such as Agile development and its associated processes, as well as the research methodologies involved throughout the entire development process.

## 2.2 Software Development Methodology

Throughout the development of this project, many notable development methodologies were employed, most prominently the Agile development method and it's related processes. It was found that this would be the most suitable for this project for a multitude of reasons, including the familiarity with the method in both theoretical experience through college and personal practical experience using the methodology in previous projects. It was understood that the project could benefit from the involved processes such as the weekly stand up meetings in which not only were any changes or developments made to the project communicated with the project supervisor but also the rationale behind the choices and changes made. This approach also benefited from a team work perspective as both members of the team were kept up to date with any potential changes, installations of new software as well as being kept up to date on how the differing advanced aspects worked. For example, explanations of why different packages or tools were used for the different aspects of AI features. The project also benefited from the frequent communication when errors occurred as they were quickly addressed and resolved by all members of the team.

Furthermore, the projects focus on a personal topic for the team meant that

the flexible and customer focused Agile methodology and greatly enhanced the development process. The ability to quickly and effectively build up a backlog of user stories and requirements helped inform the steps to be taken, as well as tests to be performed throughout the entire process from conception to implementation.

### 2.2.1 Tools and Technologies

Throughout the process, many tools and technologies were utilized that greatly assisted in enriching the development process; some of the most notable include Figma, which in the very early stages of the project was used to create a Gantt chart to map out our initial development plan, help decide features to include, and the time frame we wished to work on specific things. Furthermore, the Kanban board helped to visualize the work to be completed throughout the development process.

The tools utilized for the AI aspects of the project include **HuggingFace**, which was used throughout the development process. Initially, it was used for things such as researching possible pre-trained models which could be used in the project. Further use included conducting research into models with various desirable functionalities or capabilities as well as potential datasets to be used to refine these models relating to the overall theme of D&D and the project

Other simpler tools such as Github and the associated tools such as issue tracking were utilized due to familiarity, ease of use, and effectiveness of opening, addressing, and closing issues relating to design, code bugs or areas lacking clarity. In addition to this, extensive use was made of Microsoft Excel for the purposes of tracking the testing aspects of this project, test cases were written in detail with proper step definitions and, expected vs actual outcomes. This approach to testing was desirable and effective for the project as it made things incredibly easy to track, update and address in both weekly meetings as well as in an immediate format.

### 2.2.2 Software Design Approach

Careful consideration was taken for the approach for the software design aspects of this project as well as the advanced features implemented. A modular and component-based approach for design was utilized that kept in line with the design principles of modern software development as well as those of the Unity engine itself. Unity's entity-component system (ECS-style architecture using GameObjects and MonoBehaviours) allowed for the separation of concerns between gameplay systems such as combat logic, AI decision-making, networking, and effective User Interface (UI) component management. This approach was chosen due to the familiarity as well as its overall compatibility with the project.

A turn-based system was implemented using a state machine pattern. Each phase of gameplay (e.g., player turn, enemy turn, action resolution) was represented as a distinct state, ensuring clear transitions and maintainable logic. This approach was chosen due to its suitability for deterministic systems such as turn-based games, where control flow must remain predictable and debuggable.

For the AI system, a behavior-based approach was adopted. The enemy decision-making process was designed based on the established ideas of Finite State Machines (FSM) and Behavior Trees (BT), allowing the AI enemy boss fight to process players actions to attempt to predict their class and counter effectively. Further consideration was taken for things such as project scope and technological constraints in which it was kept in mind the models used could not be too large or demanding on hardware otherwise the project would be extremely limited. This approach was selected over more complex machine learning techniques due to it's ease of debugging, ease of interpretation, and suitability with the project scope. See section 4 for more information on the AI components.

The Networking functionality was implemented using Unity's networking framework and the related packages and dependencies (e.g., Netcode for GameObjects), with a client-server architecture. This helped to ensure the synchronization of player actions across multiple clients, this was particularly important for the turn-based design where actions and changes in state for players and enemies alike were required to be synchronized across all clients. Special care was taken to handle latency and ensure that turn order and resolution remained restricted on the server side.

Overall, the design approach prioritized modularity, scalability, and clarity, allowing individual systems (AI, combat, networking) to be developed and tested independently before integration as well as allowing the system to adapt to new changes in design or refactoring efforts.

## 2.3 Research Methodology

The research methodology for this project combined methods both in quantity and in technical terms. The primary goal of the research phase was to help inform the design of systems such as AI enemy behavior, Networking mechanics and requirements, as well as other core game mechanics.

A combination of literature review, analysis, and practical experimentation was used. A variety of sources both academic and otherwise were used to understand established approaches to concepts such as AI and game design, while existing Unity engine documentation and developer resources were used to not only inform the development process in newer areas of unity but also to help with the implementation and connection of the advanced aspects into the Unity game engine

itself.

### 2.3.1 Literature Review Methodologies

When conducting research for the literature review aspect of the project, a few restrictions and refinements were kept in mind. The papers, books, articles, and other sources used throughout the process were only used if peer reviewed, released within the last 5-7 years for relevance in time, this methodology was adhered to closely aside from a handful of sources which proved to be incredibly informative and insightful, with foundational importance in areas such as AI behavior, learning algorithms as well as Artificial Intelligence and games, in which cases an exception was made due to them being much older than other sources featured. In addition to academic sources, vital industry documentation such as Unity official documentation and networking frameworks was also utilized, as these provided practical insights into code and components which directly applied to the project.

Keywords such as "turn-based game systems," "game AI behavior models," and "multiplayer synchronization in games" were used to identify relevant materials. The sources were evaluated based on credibility, relevance, and applicability to the project.

### 2.3.2 Experimentation

During the experimental phase of the project, an iterative prototyping approach was used to evaluate different implementations of core systems. Multiple prototypes were developed to test key features such as turn-based combat flow, AI decision-making, and networking synchronization.

For the AI system, experimentation with aspects like player class information, situational actions and variations in potential data that would be fed to the systems were used to build a variety of models. These were models evaluated based on criteria such as responsiveness, accuracy, size, and computational efficiency and burden. Furthermore experimentation with the base pre-trained models was utilized to observe performance differences in models relating to complexity, memory consumption as well as accuracy. This approach proved beneficial for the project as it ensured models were built using a broad range of data derived from the game itself which ultimately increased the accuracy of the final model used.

The networking experiments focused on ensuring a consistent game between multiple clients. Tests were conducted to evaluate the synchronization of player actions, handling the turn order, and the impact of latency. Issues such as desynchronization and delayed updates were identified and addressed through our server design and an improved state validation.

Testing was conducted using predefined test cases, with results recorded against predicted results in an Excel spreadsheet. Each test case included steps, expected outcomes, and actual results, as well as an associated user story, all of this allowed for the easy identification and tracking of any tests.

This experimental approach helped ensure design decisions were evidence-based and that implemented systems were validated through practical testing rather than assumption.

## 2.4 Summary

This chapter outlines the methodologies used throughout the development of the project, including both software development and research approaches. Agile development provided a flexible and iterative framework that supported continuous improvement and collaboration. Supporting tools such as GitHub, Figma, and Excel enhanced project management and testing processes.

Additionally, the research methodology combined structured literature review with practical experimentation, ensuring that both theoretical knowledge and real-world testing informed the final implementation. Together, these methodologies contributed to a structured, efficient, and well-justified development process.

# Chapter 3

# Technology Review

This chapter is the literature review part of the dissertation and should be tightly coupled to the context and objective from the introduction. A thorough Technology Review proves that you researched what you were doing! This chapter will cover the literature review aspects of the project as well as research in general into the different components and technologies used. Furthermore it helps to identify and explain

## 3.1 Literature Review - Understanding Modern Adaptive Algorithms

Recent advances in Large Language Models (LLMs), such as those developed by OpenAI and Anthropic, have clearly demonstrated their advanced capabilities when it comes to functions such as Natural Language Processing (NLP), understanding, as well as proficient response generation. These models and their associated algorithms have been successfully applied in areas such as healthcare, where AI-assisted systems support decision-making processes Sharma et al. (2022) [1], as well as in other areas such as content generation. However, while these applications and models demonstrate strong performance in these structured environments, their integration into interactive and real-time systems, such as video games, remains mostly underdeveloped and unexplored. This presents an opportunity to investigate how LLMs and their advanced algorithms may be applied to video games to similarly enhance a user experience through adaptive behavior which forms the basis for the AI functionality of this project.

One of the most prominent applications of these advanced learning and recommendation systems can be observed within large-scale streaming platforms such as Netflix, YouTube, and Spotify. These platforms use advanced recommendation algorithms to help deliver personalized content to users, using data generated

through user interaction to inform their decision-making processes. By analyzing patterns such as viewing history, duration of engagement, and content preferences, these systems generate recommendations that closely align with previously consumed material. As outlined by Covington et al. (2016) [2], large-scale recommendation systems, such as YouTube, use a two-stage architecture consisting of candidate generation and ranking models. These systems then filter the often large libraries of content into a more manageable and relevant set of data before sorting and selecting the most relevant based on the user's own engagement metrics. This approach enables these systems to efficiently personalize recommendations while continuously adapting to user interaction patterns. Gomez-Uribe & Hunt (2015) [3] as well as Davidson et al. (2010) [4] go into detail about some of the core components involved in the recommendation algorithms used by platforms such as Netflix and YouTube. As outlined, these systems utilize a variety of information derived from both the content consumed and the user themselves, such as:

- **Content Type** - The "tags" of the current video, the language spoken, and its overall category.
- **Watch history** - Details derived from the user's account relating to previously viewed content and interaction patterns.
- **User demographic** - Information such as age, region, and potentially device or platform usage.
- **Other contextual metadata** - What could be considered "popular" or "trending" at a particular point in time, as well as session-based activity.

These features are combined to build a dynamic profile of the user, allowing the system to make increasingly accurate predictions about what content the user is most likely to engage with. As a result, the recommendation process becomes progressively more refined over time, adapting to both long-term user preferences and short-term behavioral patterns. This displays the importance and relevance of data-driven adaptation within large-scale systems, where user interaction directly influences output. This approach to advanced decision making is particularly relevant to this project, highlighting how the way users interact with an application can directly influence the decision making process and subsequent result has inspired the adaptive AI system proposed within this project.

This raises the question of *why such recommendation systems are implemented at all?* Fundamentally, their purpose is focused on maximizing user engagement and retention. By continuously analyzing user interactions, these systems build dynamic user profiles that capture preferences, behavioral patterns, and contextual factors such as location, age, or session-based activity.

These profiles enable platforms to efficiently filter large content libraries and deliver highly personalized recommendations. For example, on YouTube, a user who consistently engages with content related to a specific topic—such as cooking or video games—will experience a rapid shift in recommended content. Suggestions will move from broad and generalized by what is considered "popular" or "trending" to increasingly specialized as the system adapts to observed behavior, even in the absence of a persistent user account. This progression highlights the system's ability to infer user preferences in real time and refine its outputs accordingly.

The impacts of these systems are substantial and have proven to be incredibly valuable to modern applications which handle adapting to user behavior. Industry reports indicate that a significant proportion of content consumption on platforms like YouTube is driven directly by recommendation algorithms, highlighting their effectiveness in shaping user behavior and sustaining engagement. This reinforces the central role of recommendation systems as not merely supportive features, but as core drivers of platform interaction. YouTube's own chief product officer has addressed just how vital such systems are by explaining [5]:

> *More than 70 percent of the time you spend watching on Google's massive video site, you're lured in by one of the service's AI-driven recommendations.*

From a user experience perspective, the sense of "understanding" exhibited by these systems plays an enormous role in user retention. When users are consistently presented with content aligned to their interests, they are more likely to remain engaged for longer periods. Importantly, these techniques are not limited to video-based streaming platforms. Music streaming services such as Spotify use similar approaches, incorporating additional layers of analysis such as contextual and affective modeling.

As highlighted in (2018) [6], more advanced recommendation systems may utilize *emotion-aware* or context-sensitive models, which attempt to capture the mood or intent associated with content such as music. By aligning the recommendations with a user's emotional state, these systems aim to improve personalization and overall user satisfaction. However, there similarly exists some area of "Grayness" within the advanced techniques wishing to adapt based on arbitrary or subjective subjects such as emotion which can greatly or slightly vary from person to person, thus decreasing the accuracy of these techniques.

This body of work demonstrates how behavior-driven adaptive systems can significantly influence user interaction patterns. However, while these systems have been highly effective in content recommendation domains, their application within interactive environments such as video games remains relatively limited. This presents an opportunity to explore how similar adaptive principles can be

applied to dynamic in-game behavior. This principle of continuous adaptation based on user input informs the rationale of the design of the proposed system within this project.

While recommendation systems operate in non-interactive environments, their underlying principle of adapting outputs based on user input can be extended to interactive systems such as games. This creates an opportunity to apply similar adaptive techniques to AI-driven entities within game-play contexts.

### 3.1.1 Understanding AI in Games

The term "AI" is thrown around and used quite loosely when it comes to modern gaming applications. It is often accurately used to describe decision making models such as non-playable characters (NPCs) or enemy behaviors, and it's usage is not entirely unfounded as the simulated "intelligence" behind these virtual characters is in fact artificial. However, with how the term is used so closely with LLM's today there is understandable cause for confusion. The decision making behavior behind these entities does not boast the same complex thought process or decision making capabilities as models such as Claude. Instead, many of the "decisions" characters make are simply based on hard-coded values or choices with varying chances of being chosen or selected. For example, an enemy being controlled by classical AI systems is far more likely to walk towards the player than they are to dodge randomly. As discussed by Yannakakis and Togelius (2018) [7], the primary goal of AI in games is not to achieve true intelligence, but to create the illusion of intelligent behavior that enhances the experience of the player.

#### Classical AI

Classical AI follows the design highlighted by many like Yannakakis and Togelius in which the decision making systems behind non NPC's are driven by a few notable and still relevant control methods, some of which are used in machine learning to this day.

- Finite State Machines (FSM): one of the most widely used approaches in game AI, where an agent transitions between predefined states such as idle, attacking, or fleeing based on environmental conditions with a predefined number of actions that can or need to be taken within each state. FSM are known for being incredibly easy to use as well as being widely used within decision making frameworks until around the mid 2000's.

- Behavior Trees: Possibly the most widely known control method as its still used in machine learning practices today. It differentiates itself from FSM by focusing on behavior rather than a singular state. With a tree like structure,

a root node selects and manages its child nodes. These methods have more flexibility when it comes to how they can be designed and tested however they still suffer with some drawbacks including "Static Knowledge Representations"

Enemies, for example, would have a set or static number of "moves" or actions available to them which can be used any amount of times, usually with a small "cool down" or waiting period after usage before they can be used again. There is no way for them to fluidly take or come up with some new action as this would require breaking or potentially rewriting its own code. The decision making process impacting these actions is also somewhat static with some factors influencing the likelihood of certain moves being used such as distance from the player but for the most part these models are quite rigid and static and based on a principle of actions that make sense from a game-play perspective. An enemy entity is more likely to walk towards the player character or perform some other form of standard movement or action in some way than it is to dodge for no reason.

### 3.1.2 Applications of LLM's in Games

There are many theoretical applications and uses for LLM's within modern gaming, as outlined in Roberto Gallotta et al (2024) [8], table 3.1 describes some of the main proposed applications of LLM's in the future of gaming. Such systems enable a more dynamic and fluid approach to things like enemy decision making, content or dialog generation, analysis, or even acting as a player assistant. This stands in contrast to traditional systems that rely on things such as behavioral trees or reinforcement learning. The proposed design for adaptive or AI driven enemy behavior within this project follows this theoretical design closely while trying to limit or counteract any potential hallucinations that come with the usage of LLM's in any form of content generation. Player actions are used as input data and used in the backend scripting which in turn calls the LLM to make some form of decision relating to the situation with the particular context in mind. As stated previously, further discussion of the specific design of the system behind ai decision making will be addressed in section 4.

Although Table 3.1 highlights the potential applications of LLMs, these approaches are not without limitations. Common challenges include the lack of concrete determination in decision making, the risk of hallucinated outputs, and difficulties in testing and balancing AI-driven systems in structured game-play environments.

The subject of behavior and decision making within games today is incredibly relevant to the project, as it forms the basis for the idea of "Adaptive" enemy behavior. A term meant to describe a state within a gaming application where the

games enemies responses or behavioral patterns are directly informed by environment or circumstance.

## 3.2 Research into Development Environments

The environments in which gaming applications are developed today have broadened greatly, resulting from the increased interaction and development of video games. Multiple environments exist for a multitude of features and platforms such as RPGMaker, Unity, Unreal Engine, and many others, while similar there are a few differences that cause environments to be drastically different and can affect the types of gaming applications that can be developed within them.

There exists some variation in the capabilities of each engine, for example Unreal Engine is an industry leading development environment with an editor capable of generating cutting edge 3D content in both animation as well as games development, however it is noted that the engine itself is incredibly taxing when it comes to performance and often demands hardware of high specifications making it incredibly inaccessible. Other engines like Unity [9], [10] offer a more fluid and flexible development environment capable of developing both 2D and 3D games effectively. In addition to this, it boasts arguably the most active online support base through it's asset store which offers a variety of resources to assist in the development of applications. However, the engine itself can be quite demanding with memory requirements being higher than most other environments of this type. Video Game Insights discuses some of the details surrounding the usage of the most popular game engines today in [11], highlighting that Unity exists by far as the most popular engine among developers for its fluid and accessible environment for development. This can be further observed in figure 3.1 and a more detailed breakdown of some of the most popular engines can be seen in 3.2.

Given the requirements of this project, particularly the need for AI integration and networking support, Unity was selected due to its extensibility, large ecosystem of tools, and compatibility with external technologies.

## 3.3 Application of advanced technologies in Unity

As stated previously, the advanced features of the project focus on the areas of applying LLM or similar machine learning models to Unity, as well as incorporating networking capabilities to ensure consistency, connectivity, and further operability of the application. Fortunately, there exists a rich amount of documentation and tools to assist in the development of these features as well as to assist in the development of the game overall. These include the main unity documentation for

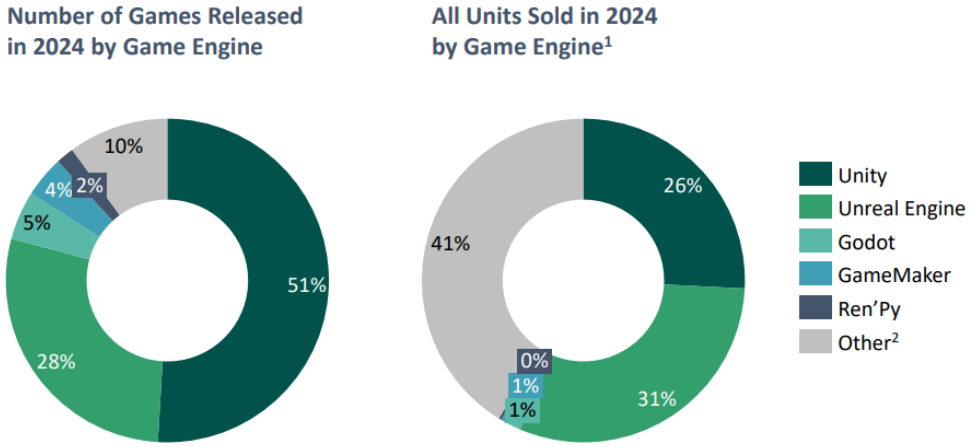


Figure 3.1: A comparison of 5 of the most popular engines for games development as recent as 2025

the development of 2D games [12] which provides great insight into the various components and systems involved in such applications, including:

- **Components** - The main components of any Unity application including scenes, panels, buttons and other operational aspects relating to both UI and unseen operational behavior such as script management.

- **Sprites & Assets** - Guides on how to build functional and interactive UI for an application using pre-installed textures or personally designed and imported ones.

- **Tilemaps** - A way of constructing a grid bases system for a variety of use cases.

- **Unity AI** - A new Unity functionality in which prompts can be used during development to create a variety of things such as sprites, textures, audio, or animations.

Alongside this, the documentation also provides insight into the best practices for the backend relating to all Unity games, the most important of which to this project include those relating to *performance optimization* as well as *scripting*. Furthermore, due to Unity's open and interactive community, the featured asset store available both on the web as well as directly through the editor environment provided a suite of assets and resources that assisted in speeding up development as well as enriching the app further.

### 3.3.1 Application of AI in Unity

Integrating LLM's directly into unity and other video game applications has previously been considered incredibly complex if not impossible due to a multitude of reasons. Primarily due to the heavy computational burden many applications face with their implementation as well as an over reliance on external services or Application Programming Interfaces (API's). However, recent research and the development of LLM deployment and the creation of specific tools have greatly increased the ease with which these models can be implemented, making advanced behavior as outlined by Gallotta et al. (2024) [8] and table 3.1 more feasible and achievable.

Recent research supports the application of Large language models being applied to games such as Unity, Sweetser (2024) [13] demonstrates that LLMs can be applied across a wide range of game development tasks, including non-player character behavior, dialogue generation, and procedural content creation. These systems enable more adaptive and context-sensitive interactions compared to traditional rule-based approaches, this further supports the proposed applications of AI within video games covered previously.

In response to these developments, a suite of tools have been developed as well as research conducted which make it increasingly practical to directly integrate an LLM into game engines such as Unity. Tools such as **Ollama** [14] provide a lightweight framework for running models locally, directly through Unity, this reduces previous issues such as latency and enables real-time interaction without reliance on external API's or services. Other tools such as **NuGet** [15] have been developed to speed up the development process, as they allow the installation of packages such as Ollama directly within the Unity editor, which has been shown to greatly reduce the time and complexity of manually installing these packages or frameworks, which could lead to errors, corruptions or incomplete functionalities due to missing complexities. These technologies were chosen for this project due to their relevance and capabilities in helping to develop the core AI functionalities.

This approach aligns with broader research on efficient model deployment. Techniques such as parameter-efficient fine-tuning, as discussed by Dettmers et al. (2023) [16], have significantly reduced the computational requirements of large models, making them viable on consumer-grade hardware. In addition, local inference offers advantages in reduced latency, improved privacy, and greater control over model behavior.

Within the context of this project, Ollama and NuGet were selected due to their ability to enable real-time interaction between the Unity application and an LLM, allowing dynamic enemy decision-making without reliance on external cloud-based services, as well as ease of use and installation.

### 3.3.2 Unity Netcode and Networking

The development of networking over the last few decades has led to the creation of increasingly complex and interactive multiplayer experiences within video games, as well as numerous architectures emerging to enable their creation and consistency. Such systems within video games focus greatly on the areas of synchronization of data and state as well as ensuring security and consistency of the game state across multiple clients. As these online aspects have come a central component in modern video games a variety of networking architectures have also emerged to support their development and application.

One of the most commonly discussed approaches is Peer-to-Peer (P2P) networking, where each client communicates directly with others without a central authority. As outlined in Fazea et al. (2021) [17], unstructured P2P networks offer flexibility and resilience, allowing nodes to dynamically join and leave the network. Similarly, frameworks such as WebRTC-based multiplayer systems demonstrate how low-latency, browser-based peer communication can be achieved in real-time gaming contexts. It differentiates itself from traditional client-server models by lacking a central authoritative server that monitors and manages the overall state of the game. Centralized systems, while useful, come with a variety of disadvantages such as costly scalability, risk of higher latency, and overall cost of hosting and maintaining servers.

However, while P2P approaches provide advantages in decentralization and scalability, they introduce challenges in maintaining consistent game state, handling latency, and preventing de-synchronization between clients. These issues are particularly relevant in structured game-play systems, such as turn-based games like the one covered in this project, where accuracy and consistency are critical. Fully decentralized architectures remain quite impractical for real-time game engines with a number of limitations such as Network Address Translation (NAT) traversal, state consistency, and security. This creates an opportunity for the project to create a hybrid approach in which the Dungeon Master's machine behaves as the authoritative host that distributes a validated and consistent game state to all connected clients.

As a result, this project adopts a client-server architecture through Unity's Netcode for GameObjects framework [18]. This approach allows for Dungeon Masters (DM's) to have authoritative control over game state, ensuring that all player actions are validated and synchronized consistently across connected clients. Unity's Netcode offers ease of access within the editor itself, as the framework can be easily installed into the project with little need for extensive or complicated configuration. Furthermore, due to the high level of documentation surrounding the framework, the process of properly setting up the P2P system within the project was greatly accelerated.

### 3.3.3 Unity Networking Tools

As discussed in table 3.2, unity offers a dynamic and flexible development environment with a range of frameworks to solve issues or implement further functionality such as networking. These include

- Unity Netcode - The most modern multiplayer framework supported by unity [18]. It utilizes tools for managing sessions, synchronizing state and objects as well as handling Request-Response Protocols (RPC) through its authoritative architecture. This proves beneficial for the project when managing all communication relating to the host and connected clients
- Unity Relay - A framework utilized to improve the reliability of connections within a session by acting as a proxy for multiplayer traffic [19]. This enables clients to connect efficiently when direct communication is blocked by firewalls observed in NAT systems.
- Unity Transport - A lower level networking tool providing a layer with greater control over data and content delivery [20]. It can easily be used in client-server setups or P2P due to it not enforcing any specific model.

## 3.4 Research Gaps

Based on the research conducted, it can be confidently stated that as of the time of writing, the majority of work primarily focuses on the theoretical applications of LLM's and other related AI technologies within video games today with experimental concepts being introduced but with very little integration into actual game engines. Little work exists relating to balancing these AI or LLM models within structured environments such as video games, attempting to restrain or restrict their unpredictable or undesirable behavior while still retaining content generation and advanced processing.

This project aims to address these gaps by implementing an LLM-driven adaptive enemy system within a Unity-based, networked, turn-based game environment, demonstrating a practical application of these emerging technologies.

Table 3.1: Applications of LLMs in Modern Game Development

| **Application** | **Description** | **Advantages** | **Project Relevance** |
|---|---|---|---|
| Player Character | Capabilities of LLM's can be leveraged to produce more efficient action-producing programs. | High adaptability with content/response generation, immersion and reduced need for pre-written or hard coded scripts. | Irrelevant |
| Non-Player Characters (NPC's) | Enemy decisions are generated dynamically based on game state and player actions. | Enriches the experience with increased unpredictability and re-playability; more engaging game-play. | Relevant as it forms the basis for adaptive enemies |
| Player Assistants | Natural Language processing and generation are used to generate scenario specific hints or tips to assist the player. | Enriches experience and helps guide player through the game. | Somewhat relevant. |
| Game Master | AI generates and manages components relating to narrative | Improves accessibility and user experience. | Irrelevant. |
| Design Automation/Assistant | Development assistant for level, music or algorithm generation. | Can assist in the development process for applications and content generation. | Irrelevant. |

Table 3.2: Comparison of Game Engines

| **Engine** | **Advantages** | **Disadvantages** | **Type** |
|---|---|---|---|
| Unreal Engine | Industry leading 3D content generation with a robust editor and cross platform support. | Steep learning curve for beginners, large burden on hardware and large delivery files. | Free (Royalty Based) |
| Unity | Incredibly strong versatility for both 2D & 3D games development, comprehensive documentation as well as an active asset store for development assistance. | No external code libraries for assistance as well as some higher memory usage | Free & Paid Tiers |
| Godot | Rapid prototyping, enabling the development of both 2D & 3D games with dedicated 2D engine. Runs efficiently on lower spec hardware | Limited native support for console development, poor performance on high end 3D graphics and smaller asset ecosystem | Free - Open Source |
| GameMaker | Easy to learn with rapid development for 2D games | Smaller community, poor 3D support and capabilities, lacks real rendering, physics or optimization tools for performance. | Free - But paid comercially |

# Chapter 4

# System Design

The following chapter will provide a detailed explanation of the system architecture, including the internal components used within Unity, the flow of data and interactions, the usage and integration of external tools and packages, as well as a discussion of the constraints of the application, error handling and validation techniques, and the potential scalability and future developments to be made.

## 4.1 Overall System Architecture

Dungeons Together was built entirely within the Unity game engine with a backend composed primarily of Monobehaviour C# scripts with GameObjects within being used to enable behavior such as navigation, player interaction as well as the networking and state management. In an attempt to solve and reduce common issues faced within P2P networks, this project adapts systems such as inconsistencies with state and privacy issues and security, this project adapts a networking system utilizing the Unity Netcode and Relay Packages. Within this system, the game host/Dungeon Master will act as the server during sessions by creating a relay session in which players can join over the network by simply entering a randomly generated alphanumeric code distributed by the host. During these sessions, the flow of data and state management ensure that the game state and its associated aspects such as actions performed, enemy and player character health updates, as well as token positioning on the game board, are saved and distributed consistently among all connected clients.

The main architecture of the game is comprised of 3 main layers:

- **Networking Layer** - This layer handles the aforementioned networking functionalities of the application. Using Unity Netcode and Relay, the state of actions is kept up to date as the host distributes the new state to all clients

after an action is successfully performed and the state of the game has safely been updated in some way.

- **Backend/Data Layer** - This layer handles the backend behavioral scripting aspects of the game. All functionalities relating to game performance such as the movement of player and enemy tokens, the performance of attacks, control and subsequent management of combat state as well as other managerial functions relating to campaigns in general are configured within this layer in the form of multiple C# scripts, which is the primary programming language supported by Unity's backend services. Further controls built with this layer include the fundamental services relating to the creation and management of player characters and personal campaigns on a users own device alongside functionalities such as user authentication and music.

- **Presentation/Frontend Layer** - This layer of the game relates to the visual aspects presented to the players during use. Items such as images representing player tokens, as well as panels for menu items and other control aspects were all designed, built, and managed in this layer. This was made especially easy due to Unity's friendly editor and inspector for configuring things such as aspect ratio, view, screen scaling, and other aspects relating to the User Interface (UI).

The game itself and it's main functionalities are driven by multiple manager scripts within the backend, with data scripts being used to generate a frame on which to build commonly used items related to game-play such as player characters, enemies, weaponry, or campaigns. These managers assist in coordinating actions within the game itself, such as loading and representing a user's saved characters from their raw JSON files or enabling the Dungeon Master to control the flow of combat in an effective and simplistic manner. A UML diagram of the entire system architecture is observed in .

## 4.2 System Interaction and State Management

The system of Dungeons Together follows a structured interaction driven flow and turn-based model which is divided into various phases categorized by functionality alongside networking. Throughout these specific areas the player performs some form of action, from creation of a new character or attacking a creature during combat, these actions are managed by the aforementioned management scripts which apply the changes to the state of the game appropriately and ensure that all relevant data is saved to the players local device.

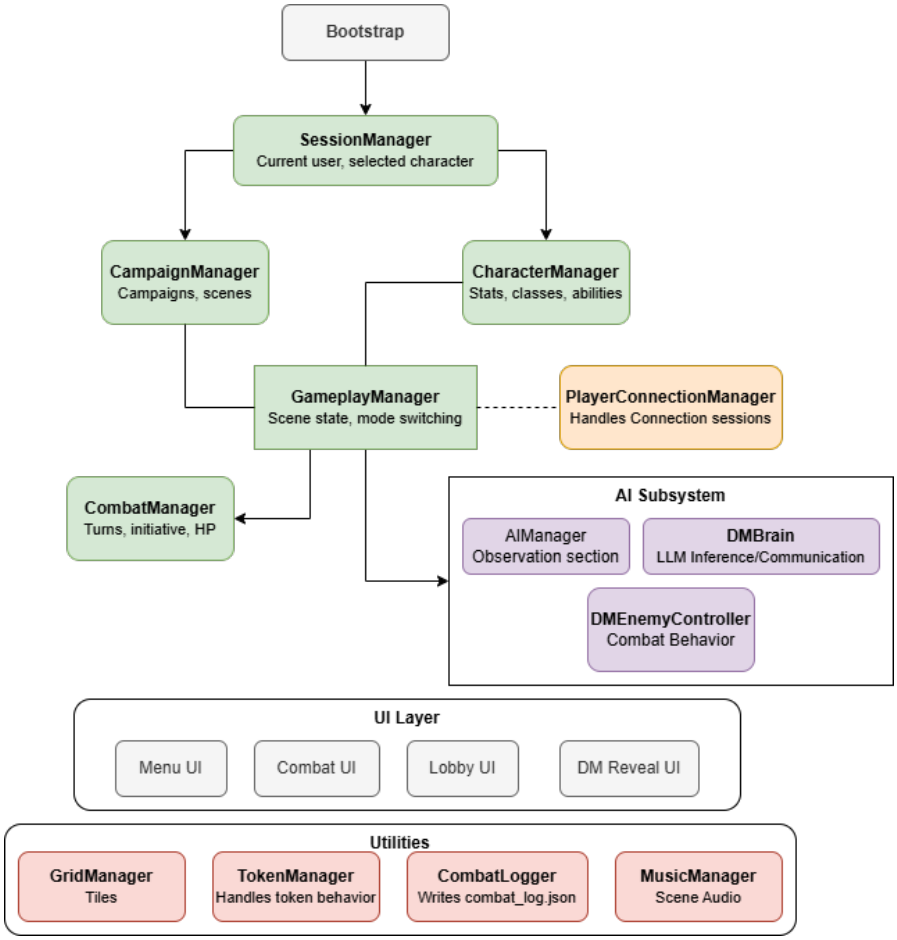


Figure 4.1: Overall System Architecture

In the case of the main combat or game-play functionalities, the networking management is utilized to greater effect. During combat, given it is the current players turn, they may perform an action of some sort such as attacking, movement or casting a spell; these are then processed by the CombatManager GameObject alongside the Networking functionalities to apply the changes made to the current game state and distribute the change throughout all connected clients.

Similar components are utilized within the AI driven Dungeon Master Boss fight. As previously covered, when a player performs some form of action, management scripts trigger certain functionalities within the backend to not only reflect the desired change on the state of the game but to also record the player's action in a specific format,

This structured flow ensures consistency across both local and networked game-play.

## 4.3 Core Components & Implementation

The system is composed of several core components that work together to provide a structured and interactive game-play experience for players. These components are implemented through Unity's component-based architecture, where functionality is distributed across modular scripts attached to GameObjects.

These components are designed to be modular and reusable, allowing individual systems such as combat, AI, networking, character, and campaign management to operate independently while still contributing to the overall state of the game.

### 4.3.1 Character & Campaign Management

Character and campaign management form the foundation of persistent state in Dungeons Together. Character files store all player-specific data required during a session, while campaign files maintain the broader game context managed by the Dungeon Master, including session state and map or environmental configuration. Although both components share some of the foundational scripting functions within the application such as serialization, storage, creation, and loading, their uses are drastically different.

### 4.3.2 Character Management

Characters are one of the most vital components to the project, being used in both the singular offline as well as networking functionalities. The main functionalities and uses relating to characters within the game include:

- Simplistic Creation - Characters may be created and then stored on the players local device in an efficient and simplistic manner in which the player only specifies the vital creative information such as stats distribution, name, class, race and uploading a relevant image to represent the characters token during game-play

- Combat/Basic Game-play - Characters are involved with every aspect of the core game-play functionalities such as movement around the map, combat when attacking other players or entities, and the networking functions in which their tokens are used to represent the most recent and updated state across all connected clients

### Data Structure & Implementation

The basic data structure of these characters is outlined by CharacterData.cs, which is one of the many data scripts in which the necessary details such as health, attributes, armor class, weaponry, and their owner-authentication name are defined for a variety of uses. These data files can be thought of as a way of defining what a certain element is such as tiles, messages and characters, an outline of the aforementioned CharacterData.cs can be seen in 4.2.

Within this figure we can see fields such as **weaponDamage** which is used to store a dice notation to be used when calculating damage during combat. For example if a characters weapon is set to "Broadsword" then their subsequent weaponDamage field will be set to "1d8" to inform the system later to apply a raondomly generated value between 1 and 8 to a target during combat. This design greatly assisted in reducing costly if statements to check weapon then apply damage appropriately, instead the dice notation is parsed to be used directly for damage.

Similarly the field **ownerUsername** is utilized for purposes of linking during an authenticated session, this effectively allows the system to only represent or retrieve characters from the local device that are owned by this specific authenticated user.

### Creation & Persistence

During character creation the user interacts with various GameObjects in the Character creation scene, when all the necessary fields have been filled out such as the characters name, class, race and their stat distributions the CharacterManager.cs script attached the a management GameObject within Unity creates a new instance of the CharacterData class. This management object then calls the method *SaveCharacterJson(string json, string baseFileName)* which assigns the ownerUsername using the SessionManager object. This object is initialized

```
[Serializable]
public class CharacterData
{
    public string id;
    public string charName;
    public string race;
    public string charClass;
    public int level;
    public int AC;
    public int HP;
    public int speed;
    public string weapon;
    public string weaponDamage;
    public int proficiencyBonus;
    public int strength, dexterity...etc.
    public Dictionary<string, int> spellSlots;
    public List<string> abilities;
    public string tokenFileName;
    public string createdAt;
    public string ownerUsername;
}
```

Figure 4.2: Outlines the CharacterData.cs structure used during character creation

upon the game's start up and helps to manage the user authentication throughout the entire session, within this file the *SessionManager.Instance.CurrentUsername* method is called to retrieve the current username to be assigned to the created character. Finally the SaveCharacterJson method calls the SaveAllCharacters() method to serialize the newly created character and save them in the JSON format.

This approach was selected due to its flexibility, and strong compatibility with Unity's data handling systems, allowing a system that is both scalable for adding new data structures for additional features later, but also easy to modify. From here characters may be loaded in an viewed from the main character screen where they may be viewed or edited, during which saving will result in the same methods being called to update the character file. An outline of CharacterManager's creation can be seen in the figure 4.3 and the resulting created JSON file can be seen in figure 4.4. These figures outline how the structure explicitly defined within the data files is adhered to when creating and subsequently storing these files to the players device.

```
// Save JSON text to file
public static string SaveCharacterJson(string json,
string baseFileName)
public CharacterData CreateCharacter(string name,
string race, string charClass)
{
    CharacterData newChar = new CharacterData
    {
        charName = name,
        race = race,
        charClass = charClass,
        ownerUsername = SessionManager.Instance.CurrentUsername
        // ...other fields initialized as needed
    };
    allCharacters.Add(newChar);
    SaveCharacters();
    return newChar;
}
```

Figure 4.3: Overview of the Character Creation method used by the CharacterManager.cs script

**Loading & Session Use**

After creation, editing and basic representation the flow of character interaction leads to the loading or session usage functionalities. The SessionManager method SetSelectedCharacter(CharacterData character) is used during the initially stages of players participating in a campaign. Initially upon joining a scene, players are given a selection of their saved characters to use in the session, upon selecting the SessionManager GameObject within the scene calls the aforementioned method which simply sets the selectedCharacter variable to be the data relating to the chosen character. Following the character being set in the backend, when the host of the game requests the characters from clients, the relevant character data relating to the players chosen character along with their authentication name is returned. This is accomplished with a combination of the CampaignManager object's AddPlayerToCampaign and the various assignment methods within the PlayerConnectionManager GameObject and script. Greater detail relating to the networking aspects will be discussed in section 4.5.

```
{
    "id": "a64b1eb6-5d84-4395-80c1-ccf84e4b1a6d",
    "charName": "Chainsaw Man",
    "race": "Human",
    "charClass": "Paladin",
    "level": 20,
    "AC": 15,
    "HP": 205,
    "speed": 30,
    "weapon": "Longsword",
    "weaponDamage": "1d8",
    "proficiencyBonus": 4,
    "strength": 21,
    "dexterity": 21,
    "constitution": 21,
    "intelligence": 21,
    "wisdom": 21,
    "charisma": 21,
    "abilities": [
        "Lay on Hands"
    ],
    "tokenFileName": "Chainsaw Man_20260402103334_token.png",
    "createdAt": "2026-04-02T10:33:34.6798623Z",
    "ownerUsername": ""
}
```


Figure 4.4: An example of a serialized character file as stored on a player's local device

### 4.3.3 Campaign Management

Campaigns similarly exist as one of the core functionalities of Dungeons together, it plays the vital role of facilitating the online aspects in which a singular player loads a saved campaign from their device, start's a session and distributes a code so that players may connect automatically. While their initial creation, serialization and de-serialization techniques may be similar to characters, campaigns act as more of a container for specific scenes to be played in sessions as opposed to a singular object used within the game-play itself, facilitating things such as player assignments to characters as well as hosting the sessions themselves. The main elements of the Campaign management system include:

- **Simplistic Creation** - Similarly to chatacters, campaigns can be created, altered, deleted and stored on the players local device. Initially the player only provides aspects such as a name, description and logo for the campaign to allow it to be easily recognizable.

- **Network Management** - Upon creation, the player may choose to host this campaign through the use of the Network systems. A alphanumeric code may be created and distributed so that players may join, distribute the character they wish to play as, then begin to play out whatever scene is selected. This plays an essential role in the synchronization of data and consistency in state, the design and implementation of networking functionalities will be further discussed in greater detail in section 4.5.

- **Scene Management** - Within a campaign exists multiple scenarios (scenes) in which the connected clients may play out a variety of combat situations or simply role play a situation set up by the hosting Dungeon Master. Further vital management components exist within this function as well, such as the generalized game-play manager scripting, combat management, weapon or spell attacks, map generation, token movement, and combat logging which plays a vital role in the AI functionalities which will be discussed further in section 4.4.

An overview of the complete flow of actions across both the character and campaign management systems, from initial creation through to session use 4.5.

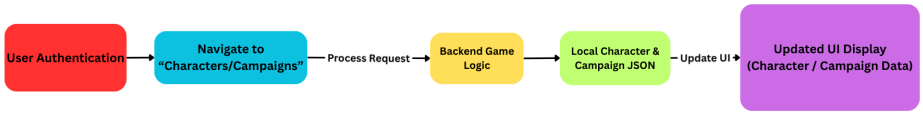


Figure 4.5: An overview of the basic flow of actions for campaign and character interaction

### Data Structure & Implementation

As previously discussed, the purpose of campaigns as a structure within the game is to effectively create a structure in which multiple scenes relating to a campaign may be stored and managed accordingly. This is accomplished through the use of multiple of the aforementioned data structure files, specifically CampaignData.cs and SceneData.cs which map out the required components to build a Campaign as well as it's associated scenes. An outline of both the campaign and scene data structures can be observed in figures 4.6 and 4.7.

```
[System.Serializable]
public class Campaign
{
    public string campaignId;
    public string campaignName;
    public string campaignDescription;
    public string dmUsername;
    public List<PlayerCharacterAssignment> playerCharacters;
    public List<SceneData> scenes;
    public string inviteCode;
    public string createdDate;
    public string campaignLogoPath;
    //..Related Methods
}
```

Figure 4.6: An overview of the campaign data class used to outline it's elements

```
[System.Serializable]
public class SceneData
{
    public string sceneId;
    public string sceneName;
    public SceneType sceneType;
    public string description;
    public string lastPlayed;
    public string status;
    public string sceneSnapshotPath;
    public MapData mapData;
    public List<string> activeCharacterIds;
    public List<TokenData> tokens;
    //...Related Methods
}
```

Figure 4.7: Similarly highlights the scene data class used to outline the elements of a scene

It can be noticed that within CampaignData there exists the variable public List<SceneData> scenes;, this nesting is the key to campaign and scene correlation as it enables the ease of actions such as the addition, deletion of scenes within each campaign instance.

**Creation & Player Assignment**

Given the player is authenticated correctly, upon navigating to the campaigns page they will be prompted with the option to create a new one, similarly to character creation the player can then fill out the necessary fields only such as the name and description as well as a logo for the campaign itself. Following this, upon submitting the entered details, the CampaignManager instance calls public Campaign CreateCampaign(string campaignName, string dmUsername, string description = ""). During which the appropriate fields are set using a CampaignData instance then saved to the users device using a similar serialization method as observed in the character saving process.

When a new campaign is created, its scenes list is empty and the Dungeon Master may create new ones using the methods within CampaignManager.cs of AddScene() which simply adds a blank scene to the campaign's data and AddSceneWithData() which is called following the creation of the map, during which the scene and all relevant map configuration data such as tile type and size are saved to the main campaign's data. An outline of these methods within the CampaignManager.cs can be seen in 4.8.

Following the creation of both campaigns and their relevant scenes, the Dungeon Master may begin a session which players can join. During this, as previously mentioned when a player joins the session, the Dungeon Master may add the specific characters sent by clients to the current campaign. This is accomplished using the GameplayManager's AddPlayerCharacterToCampaign() method, in which the player's username and network ID are linked to their respective CharacterData, following this the upated state is saved. This highlights the point where the character and campaigns systems converge and connect. Characters while also being their own vital element also form one piece of the Campaigns functionality.

```
public Campaign CreateCampaign(string campaignName,
string dmUsername, string description = "")
{
    currentCampaign = new Campaign(campaignName,
    dmUsername, description);
    SaveCampaign();
    Debug.Log($"Campaign created: {campaignName}
    with code: {currentCampaign.inviteCode}");
    return currentCampaign;
}
```

Figure 4.8: An example of a serialized character file as stored on a player's local device

```
public bool AddPlayerCharacterToCampaign(string
playerUsername, ulong networkId, CharacterData character)
{
    if (currentCampaign == null)
    {
        Debug.LogError("No active campaign on host!");
        return false;
    }

    //We then just assign the character to the player
    within the campaign
    currentCampaign.AssignCharacterToPlayer(playerUsername,
    networkId, character);
    //Then make sure to save the campaign data
    SaveCampaign();
    Debug.Log($"Player {playerUsername} joined
    with character {character.charName}");
    return true;
}
```

Figure 4.9: An example of a serialized character file as stored on a player's local device

### Core Gameplay Management

The core game-play component consists of a variety of management GameObjects used to ensure a structured execution of actions within the game itself but also to create a scalable system in which new features may be added, altered or removed. Software principles and best practices such as singletons and abstraction were applied heavily to the areas of management throughout development, this was done to ensure consistency within game-play as multiple management scripts operating simultaneously for things such as networking or campaign management could easily cause disruptions or inconsistencies in data, potentially leaving the system in an unstable state. The decision to isloate behavior based on the section of game play handled greatly helped in regards to debugging and development as functionalities relating to starting or stopping combat could only be found within the *CombatManager.cs* GameObject. The table 4.1 provides an overview of the most prominent management GameObjects utilized throughout the game.

Table 4.1: Overview of Core Management Scripts

| Script | Primary Responsibility |
|---|---|
| GameplayManager | Oversees scene setup, map instantiation, token placement, and state synchronization across connected clients. |
| CombatManager | Controls turn order, combat flow, and coordinates attack actions and applying damage. |
| WeaponAttackManager | Calculates weapon attack rolls and damage, applies modifiers, and updates character or enemy HP. |
| CombatLogger | Records and displays a chronological log of combat events for all players and the Dungeon Master. |
| CampaignManager | Manages campaign lifecycle — creation, saving, loading, and networking synchronization. |
| CharacterManager | Handles character creation, persistence, and availability for combat and campaign use. |
| SpellChoiceManager | Presents available spells during combat and integrates with CombatManager for casting logic. |

### Combat Management

The flow of combat plays a vital role in Dungeons Together, forming the basis on which connected player characters can combat against enemy characters controlled by the Dungeon Master. Combat is comprised of several components which each boast the previously discussed isolated management GameObject behavior to handle specific areas or pieces of functionality, esuring maintainability and a separation of concerns for a modular system.

**Turn Order and Initiative** Combat encounters are structured around a turn-based system. The `CombatManager` is responsible for determining initiative order, tracking whose turn it is, and advancing the round.

```
// Example: Advancing to the next turn
public void NextTurn() {
    currentTurnIndex = (currentTurnIndex + 1) %
    initiativeOrder.Count;
    var currentPlayer = initiativeOrder[currentTurnIndex];
    // Notify players and update UI
}
```

**Weapon Attacks** The `WeaponAttackManager` handles the logic for resolving weapon-based attacks, including hit/miss calculations and damage application.

```
// Example: Resolving a weapon attack
public void PerformAttack(CharacterData attacker, CharacterData
target, Weapon weapon) {
    int attackRoll = RollD20() + attacker.proficiencyBonus +
    weapon.attackBonus;
    if (attackRoll >= target.AC) {
        int damage = RollDamage(weapon.damageDice) +
        attacker.strength;
        target.HP -= damage;
        CombatLogger.Instance.Log($"{attacker.charName} hits
        {target.charName} for {damage} damage!");
    } else {
        CombatLogger.Instance.Log($"{attacker.charName} misses
        {target.charName}.");
    }
}
```

**Spell Casting** The `SpellChoiceManager` manages spell selection and casting, ensuring that players can only cast spells they have available and that spell slots are correctly tracked..

**Combat Logging** The `CombatLogger` records all significant combat events, providing a clear history for both players and the Dungeon Master.

```
// Example: Logging a combat event
public void Log(string message) {
    combatLog.Add(message);
    UpdateCombatLogUI();
}
```

**State Synchronization** All combat actions and state changes are synchronized across the network using Unity's Netcode for GameObjects. This ensures that all players see the same combat state, including turn order, HP changes, and combat log updates.

**Handoff to Networking**

At this point during the flow of campaign interaction, the Dungeon Master has the ability to not only create campaigns but also add newly joined players and their respective characters to it. This marks the point in which the base networking functionalities begin to be fully utilized through the GameObject and subsequent backend logic of *PlayerConnectionManager.cs*. This begins a takeover of tracking the main state and ensuring consistencies and synchronization of state across all clients. Notable methods utilized at this stage include OnClientConnected() which triggers when the host detects a

At this point during the flow of campaign interaction, the Dungeon Master has the ability to not only create campaigns but also add newly joined players and their respective characters to it. This marks the point in which the base networking functionalities begin to be fully utilized through the GameObject and subsequent backend logic of *PlayerConnectionManager.cs*. This begins a handover of tracking the main state and ensuring consistencies and synchronization of state across all clients.

Notable methods utilized at this stage include *OnClientConnected()*, which triggers when the host detects a new player joining the session. Upon connection, the networking manager registers the player, assigns them a unique network ID, and synchronizes relevant campaign and character data to all clients. This ensures that every participant has an up-to-date view of the campaign state.

This handoff ensures that from this point forward, all changes—such as new players joining, character assignments, or campaign updates, are distributed across the network, maintaining a consistent and synchronized game state for all participants.

## 4.4 AI Architecture

The AI functionality within Dungeons Together is designed to create an experience in which the user's own actions directly inform an enemy decision making process. Operating using a pre-trained model the system is hosted almost entirely on the users own device the architecture enables the model to interact with the scripting logic utilized by the Dungeon Master boss fight.

Accessed after the player is authenticated and navigates to the campaigns menu, they are prompted with the option to partake in an optional challenge against an enemy referred to as the "Dungeon Master" (DM). During this fight, the player selects one of their saved characters to play as before they're taken to a specific scene to fight against the DM, all featues from combat are observed here such as the main combat and attack management systems and their respective

```
// Example: Handling a new client connection
public override void OnClientConnected(ulong clientId)
    {
    // Register the new player
    connectedPlayers[clientId] = new ConnectedPlayer(clientId,
    ...);

    // Synchronize campaign and character data to all clients
    SyncAllPlayers();
}

private void SyncAllPlayers()
{
    // Sends the current list of players and their characters
    to all clients
    foreach (var player in connectedPlayers.Values)
    {
    // Custom logic to broadcast player data
    }
}
```

Figure 4.10: An overview of the player synchronization methods used to ensure consistency of state during game-play

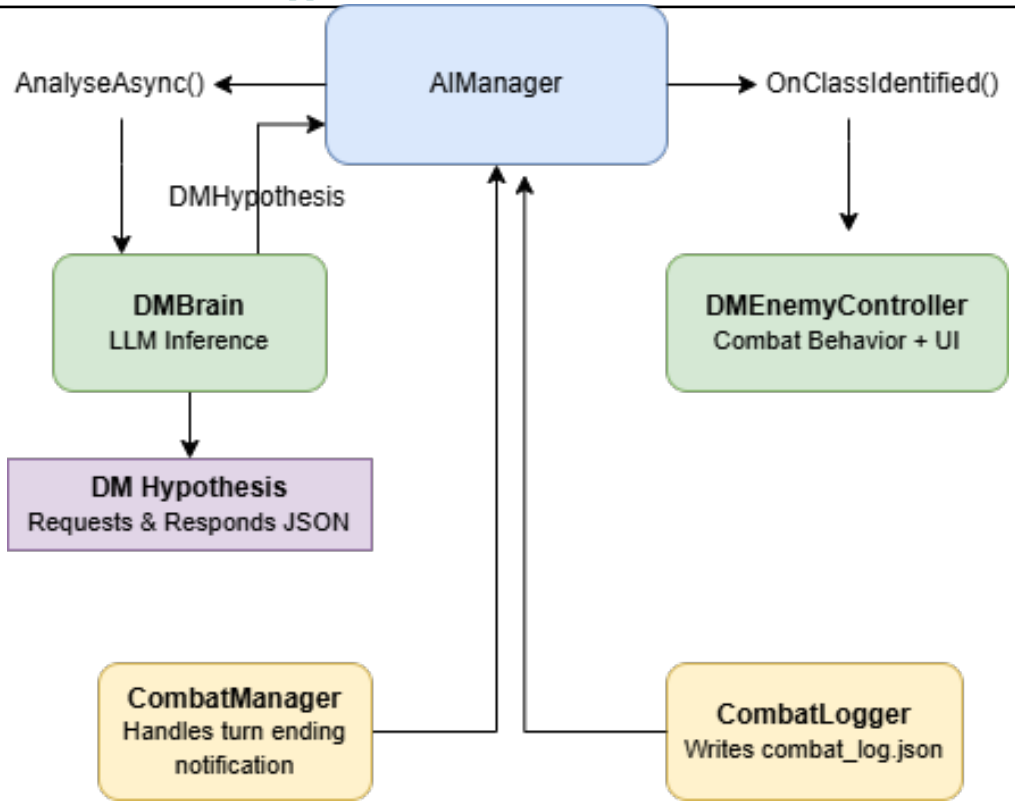


Figure 4.11: An Overview of the AI architecture

tracking and logging components, alongside the various management GameObjects relating to movement, token's, general game-play and music.

Within this game mode, the player's actions are iteratively added to a new log specifically used by the new AIManager.cs GameObject, these actions are then utilized by the pre-trained LLM, which attempts to hypothesize the player's chosen class. Upon correctly guessing the players class, the DM is given the ability to instantly end the fight and the appropriate message is displayed to the user. This section will cover this functionality within the project, as well as it's involved components and the respective training utilized to achieve the final model.

The architecture is composed of three tightly coupled components: AIManager, which coordinates observation and decision-making; DMBrain, which interfaces with the locally-hosted LLM via LLamaSharp; and DMEnemyController, which translates hypotheses into in-game consequences. These components form a uni-directional pipeline from observation through inference to action. An overview of this architecture is highlighted in 4.11.

```
def roll_dice(dice_str):
    num, sides = dice_str.split('d')
    return sum(random.randint(1, int(sides)) for _ in range(int(num)))
```

Figure 4.12: An example of one of the methods used within the training data generation, used to randomly generate dice values to ensure variation in data.

### 4.4.1 Selection, Training & Installation

**Model Selection & Training Data Creation** HuggingFace was used during the design and research phased of the project to investigate potential viable pre-trained models to be used to perform the functionalities required. This proved beneficial due to the high levels of documentation and the enormous availability of models. An overview of 3 selected models can be seen in table 4.2. For the purposes of the project, a model with strong abilities related to natural language processing (NLP) as well as low storage and memory consumption, was required. Ultimately, the model chosen and used within the project was Qwen2.5-0.5b-Instruct as it combined the smaller design and lower memory consumption aspects that were desirable for this project.

Due to the fact that the model was general purpose, its knowledge of topics related to D&D and the data generated by the project were incredibly limited. This led to the decision to investigate and utilize datasets related to D&D to refine the knowledge of the models. HuggingFace used further in this area as it supplies a plethora of datasets for public use, table CITE TABLE highlights a selection of models discovered during this phase of the project that were considered for usage. Ultimately, due to compatibility and relevance, the decision was made to generate unique training data based on the data featured within the game. This was accomplished through the use of an external python script 4.12 that compiled the classes featured within the game into more than *3,600 synthetic training examples*. This approach was used to ground the models knowledge in elements featured within the game and to reduce hallucinations.

**External Training & Integration with Unity** Google Colab was used extensively to train models after the selection process and the creation of training data. This was chosen due to limitations in local development hardware and the overall accessibility of Google Colab. During training, the model and relevant training data were configured and then trained. During the first iterations of the training data and scripting, there was only around 500 test cases fully developed, this resulted in an average training time of 15 minutes. Then in the later stages of the project, when the training samples reached 3,600, the training time increased

```
#CELL 2: Loading the models
from unsloth import FastLanguageModel
import torch

MAX_SEQ_LENGTH = 768

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name      = 'Qwen/Qwen2.5-0.5B-Instruct',
    max_seq_length = MAX_SEQ_LENGTH,
    dtype           = None,
    load_in_4bit    = True,
)

model = FastLanguageModel.get_peft_model(
    model,
    r = 16,
    target_modules = ['q_proj','k_proj','v_proj','o_proj',
                      'gate_proj','up_proj','down_proj'],
    lora_alpha = 16,
    lora_dropout = 0,
    bias = 'none',
    use_gradient_checkpointing = 'unsloth',
    random_state = 42,
)
print('Model loaded.')
```

Figure 4.13: Provides an overview of the cell used within colab to load the desired model directly from HuggingFace.

steadily to 20 minutes on average. Training loss was examined closely during this time to ensure the model would not turn out to be overfit. From here, the model was converted to a .gguf file to be utilized by LlamaSharp's inference system when downloaded and imported into Unity. From here the model was simply saved to colab's environment and downloaded onto the local development device, which took roughly 5 minutes. An example of a cell used during model training within Colab can be observed in 4.13.

Following training, LlamaSharp package was installed in the main system via NuGet's package management software. This package was chosen specifically due to it's capabilities relating to on device inference with local LLM models without reliance on external API calls, this assisted in preserving privacy and enriching

the offline capabilities of the application. The main dm_boss_f16.gguf was then imported to the Assets/StreamingAssets folder within the main application. This design decision was made to ensure easy access to the model as well as to ensure that it would be shipped with the main build during compilation and distribution.

### 4.4.2 AI Decision Pipeline

As previously highlighted three scripts handle the logic relating to the AI functionalities, DMBrain, DMEnemyController and AIManager. This results in the following pipeline for the AI behavior:

1. When a player signals the end of their turn, the CombatManager GameObject calls AIManager.NotifyPlayerTurnEnded() to trigger the rest of the AI's functionalities.

2. The combat_log.json stored within the application is then read by AIManager.ParseCombatLog(), information relating to the players actions are then extracted from this list, such as weaponry, spells, or abilities used alongside the total damage dealt. Within the manager script the variable KnownClassAbilities is then used as a assistant whitelist to separate abilities from spell casts (e.g. Divine Smite routes to spells_cast, not abilities_used). This helps to prevent hallucinations.

3. These observed actions are then constructed into an observation JSON using BuildObservationJson() which serializes the state into a structured block to be sent to the main LLM.

4. DMBrain.AnalyseAsync() then formats the observed state into a ChatML prompt and runs runs StatelessExecutor (so no conversation memory leaks between turns), and early-exits once a complete JSON object is detected from the model.

5. HandleHypothesis is then applied to the model's response to create a dual threshold: minimumConfidenceToAct (0.55, soft gate) and lockConfidenceThreshold (0.80, locks class permanently). This assists in signaling the system about the confidence of the model's class prediction.

6. The consequence of the model's output is then applied to the logic of the game. DMEnemyController.OnClassIdentified() is triggered to add 2.5× damage multiplier to the DM's attacks as well as displaying a UI panel to signal to the player their class has been discovered.

```
// Matches the system prompt in dnd_training.py exactly
    private const string SYSTEM_PROMPT =
        "You are the Dungeon Master, a powerful boss in a D&D 5e
        encounter. " +
        "Each turn you observe the player's actions, their
        ability, weapon, and any spells. " +
        "Your goal is to identify their class as quickly and
        accurately as possible. " +
        "Once you are confident, declare your hypothesis. " +
        "Respond with valid JSON only containing:
        class_hypothesis, confidence, and reasoning.";
```

Figure 4.14: Provides an overview of the initial system prompt which is sent directly to the DM brain model.

### 4.4.3 Prompt Design

Prompts were designed with the following format to match the training formatting exactly, this step is critical for fine-tuned models as it ensures consistency and reduces the risk of hallucinations. The DMBrain.cs management object handles the majority of the prompting and LLM communication, initially creating a system prompt which serves as the foundational instruction set for the LLM, establishing its role as a D&D Dungeon Master focused on class identification through combat observation. The prompt explicitly constrains output to JSON format only, this system prompt can be seen in figure 4.14.

This constraint ensures structured machine-readable responses by explicitly limiting the LLM to JSON output containing only three fields: class_hypothesis (string), confidence (float), and reasoning (string). The "valid JSON only" directive prevents verbose natural language responses that would require complex parsing, enabling a reliable interpretation of AI decisions. The observation data fed to the LLM follows a structured JSON schema that compiles player actions across multiple turns rather than providing per-turn snapshots, this process can be observed in figure 4.15.

The cumulative approach reduces noise by providing the LLM with a broader context of player behavior patterns. Rather than isolated turn-by-turn actions that might be influenced by situational actions, the system accumulates evidence across turns_observed, allowing the AI to identify consistent class indicators (e.g., a Paladin's repeated use of "Lay on Hands" or divine smite) while filtering out single-turn decisions.

Furthermore, within the system exists the temperature parameter which is used

```
//Observation builder - helps to build up a structured JSON to
send the the AI of actions performed
private string BuildObservationJson()
{
    return "{\n" +
           $"  \"turns_observed\": {turnsObserved},\n" +
           $"  \"abilities_used\":
           {ToJsonArray(abilitiesUsed)},\n" +
           $"  \"spells_cast\": {ToJsonArray(spellsCast)},\n" +
           $"  \"weapons_observed\":
           {ToJsonArray(weaponsUsed)},\n" +
           $"  \"total_damage_dealt\": {totalDamage},\n" +
           $"  \"total_healing_done\": {totalHealing}\n" +
           "}";
}
```

Figure 4.15: Displays the construction of the observation prompt within the system.

```
[Tooltip("Low = consistent JSON. 0.15 recommended for fine-tuned
classification.")]
[Range(0.05f, 1.0f)]
public float temperature = 0.15f;
```

Figure 4.16: Displays the temperature parameter as configured within the DM-Brain GameObject.

to reduce unpredictable aspects of the ai such as creativity to construct a more consistent output and overall assists in reducing hallucinations during execution. The temperature value within the game is set to 0.15 to maintain the priority of consistency, the exact implementation within the system can be seen in figure 4.16.

### 4.4.4 Integration with Combat Systems

The AI subsystem does not operate in isolation — it is tightly coupled to the existing combat structure through a series of carefully designed integration points, each of which maintains a separation of concerns to avoid introducing dependencies that would destabilize the broader system.

The primary trigger for AI activity originates in `CombatManager`, which calls `AIManager.NotifyPlayerTurnEnded()` at the conclusion of every player turn.

This keeps the AI pipeline smaller and more passive by design: `AIManager` performs no polling and takes no action unless explicitly notified, ensuring it cannot interfere with turn flow or timing.

Rather than receiving action data directly from the combat systems that generate it, `AIManager` reads from the `combat_log.json` file written to disk by `CombatLogger`. This file-based decoupling means that neither system needs a direct reference to the other. `CombatLogger` continues writing to its log as normal, and `AIManager` reads from it independently via `ParseCombatLog()`. This approach also means that the AI's view of events is always derived from the same persisted record used elsewhere in the game, avoiding duplication or inconsistency in what is observed.

Parsing the combat log introduces one important challenge: `CombatLogger` records all action types using a `"Used name"` format, without distinguishing between class abilities and spells triggered through ability use. To resolve this, `AIManager` maintains a static `KnownClassAbilities` whitelist — entries matching the set are routed to `abilities_used`, while everything else (such as *Divine Smite*) is treated as a spell and directed to `spells_cast`. This separation matters because merging the two categories would reduce the quality of the observation JSON sent to the model.

Once `DMBrain` returns a hypothesis above the lock threshold, `AIManager` must locate the DM enemy token in the active scene. Rather than holding a direct reference which would require scene-specific wiring and break if the token were destroyed and respawned, it resolves the `DMEnemyController` at runtime by iterating over `CombatManager`'s initiative order and returning the first participant whose token is of type `CharacterType.Enemy`. The identified controller then receives `OnClassIdentified()`, triggering the damage multiplier and the in-game UI response.

All three core AI components (`AIManager`, `DMBrain`, `DMEnemyController`) follow the singleton pattern consistent with the rest of the game's manager objects, ensuring they can be accessed globally without scene-specific wiring while remaining straightforward to reset between encounters via `ResetForNewCombat()`.

### 4.4.5 Model Constraints

During the application development process, the design of the AI functionality and it's associated elements was performed with multiple considerations and constraints in mind. As discussed during the research phase, heavy computational burdens as well as an over reliance on external services or API's limits the implementation of AI or LLM aspects in gaming applications. As a result, the specific AI model utilized within this game was chosen to solve many of these problems.

The choice to host the model locally was made to solve the aforementioned issue of non reliance on external services as well as addressing concerns such as

privacy when processing requests, latency when processing requests and generating responses. Further, other concerns relating to memory or RAM consumption were addressed during the model selection process, as Dungeons Together aims to be a readily accessible application which can be played across a variety of desktop's it was a requirement to select a model with lower computational requirements such as overall model size, ram consumption and latency, whilst still ensure the model generates accurate responses during game-play. An overview of 3 of the models experimented with during the design process can be seen in table 4.2.

Table 4.2: Comparison of Models during Design Phase

| **Model** | **Size** | **RAM Required** | **Response Time** |
|---|---|---|---|
| Qwen2.5-0.5 Instruct | Roughly 400MB | 1.5GB of RAM | 1-3 seconds |
| SmolLm2-1.7B | Roughly 1GB | 3GB of RAM | Roughly 10 seconds |
| Phi-3.5-mini 3.8B | 2.4GB | Between 5-6GB of RAM | Roughly 20 seconds |

The table highlights 3 proficient models selected from HuggingFace during development, each have similarities in terms of storage size and basic RAM requirements, the most drastic deviations arose in their response times. Regardless of it's accuracy during training, the Phi model proved to be the slowest in terms of latency being around 15-20 seconds, which for the purposes required within the project of informing an enemies behavior meant it was the least desirable choice. The remaining models were compared using their accuracy scores, training loss as well as overall cost in terms of storage. An outline of the training loss documented during training can be seen in 4.17.

Overall due to it's smaller size as well as it's response time and accuracy during the training phase, the Qwen2.5-0.5 Instruct model was selected for Dungeons Together. It was found that this model held the lightest computational burden whilst retaining vital functionalities required for the AI within the application, such as natural language processing as well as content generation in response to dynamic prompts. Further, it's size of only roughly 400MB allowed it to easily integrate into the main application without drastically increasing the required storage size. The greatest trade off observed within the selected model was it's need for a greater amount of training data and enhanced restrictions due to it's more generalized processing abilities when compared to the higher specification models.

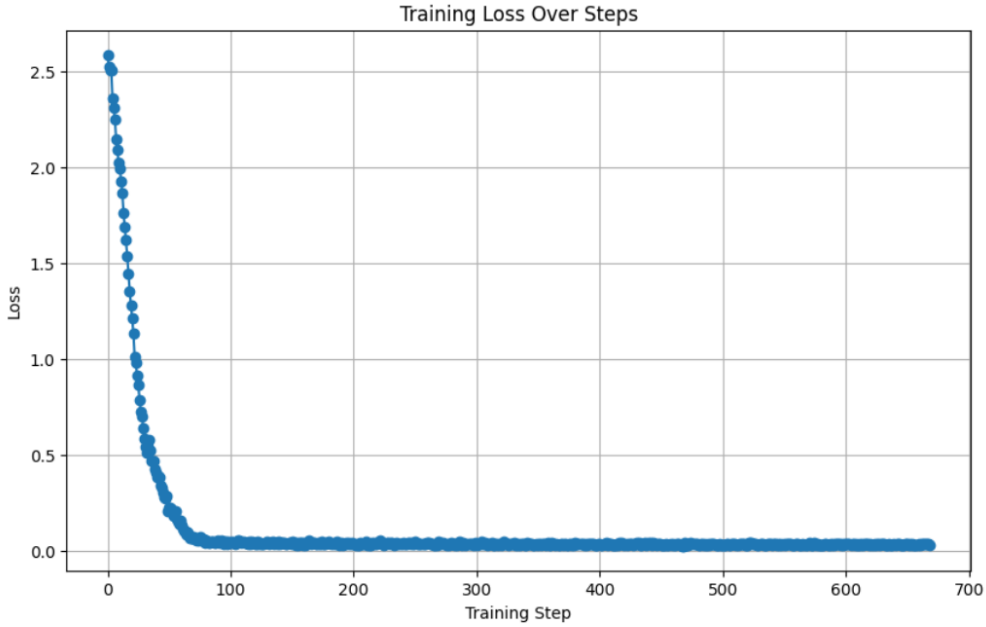


Figure 4.17: Highlights the training loss documented while training the Qwen2.5-0.5 model

## 4.5 Networking Architecture

As outlined in the Technology review 3, a common barrier for P2P systems within multiplayer games is NAT traversal. The aim of the networking implementation within this project as a result is to create a hybrid approach to networking within a multiplayer environment to solve these problems while still creating an interactive and synchronized experience for the players. Unity Relay was chosen as one of the many tools implemented to solve this issue with all traffic being routed through one of Unity's dedicated relay servers during game-play. The main architecture of the networking is composed of the following frameworks and packages; Netcode for GameObjects (v2.7.0), Unity Transport as well as Unity Services Multiplayer.

Together within the project these components are designed to support real-time multiplayer gameplay, chat communication, and synchronized game state across clients and the host. The architecture leverages Unity's networking stack (such as Unity Netcode) and custom manager classes to coordinate player actions, combat turns, and messaging. The key components involved within this section include:

- **ChatManager & ChatNetwork**: The chat system enables players to communicate in real time. ChatManager handles the user interface and message display, while ChatNetwork manages the transmission of chat messages over the network. Messages are sent to all connected clients and displayed in the

chat UI. Persistent storage is handled by ChatMessageStore, allowing chat history to be loaded when scenes change.

- **CombatManager & PlayerConnectionManager**: Combat actions and turn order are synchronized across all players. CombatManager manages the initiative order and notifies the AI and DM enemy when turns change. PlayerConnectionManager handles player connection events and remote procedure calls (RPCs), such as requesting to end a turn, ensuring that only the correct player or the DM can advance the game state.

- **SessionManager**: Manages player authentication and session state, ensuring that each player's identity is consistent across networked sessions. Always called and initialized upon the app staring up

- **Scene & Campaign Synchronization**: Scene data and campaign progress are synchronized using networked events and file-based state sharing. When a scene is loaded or changed, the relevant data is broadcast to all clients to ensure a consistent experience.

## 4.6 Constraints and Design Considerations

During the production of the game some key decisions were made relating to constraints to be placed on the application relating to functionalities as well as technologies used throughout development. These considerations include the choice of game engine, game design/structure, hardware specifications, as well as model selection.

### 4.6.1 Engine Selection & Game Design

As previously covered in table 3.2, the Unity game engine offers an extensive and dynamic environment with a plethora of tools available for developers to create games in a variety of settings. During the initial design phases of the project Unity arose frequently as the potential engine of use due to these advantages and was ultimately selected due to the decision to develop the project with a 2D perspective in mind.

Even though unity offers a suite of well developed tools to enable the creation of high performance games based in a 3D environment, these can often be expensive in terms of storage space required both during development as well as during deployment on player devices. Further, there also exists the computational and performance based burdens with such systems, as higher specification hardware would potentially be required to render any environments created, combined with

| Story Number | Story Description | Test Steps | Test Type | Acceptance Criteria | Actual Results | Eoins Notes | Story Value/Points (*/10) |
|---|---|---|---|---|---|---|---|
| 1 | As a dm, I wish that my players could connect to my session from different devices on different networks so we can play anywhere at any time | 1. I connect on my network, Cian connects on his<br>2. One hosts game and other uses code to join waiting room<br>3. Both send Messages to see if it works across different networks | Networking | Both of us can join and messages can be sent regardless of location | Success | Works fine, just need to ensure multiple people can join by testing on multiple devices | 10 |

Figure 4.18: Displays an example of the testing and validation documentation kept throughout the development process

the already established increased memory consumption [9] lead the development in the direction of a 2D perspective. This enabled an overall development process in which scenes or campaigns may be created and experimented with exponentially faster, furthermore due to graphical fidelity not being an area of focus, a more stylized and simplistic 2D focus was selected. An additional benefit from this approach was a resulting lower hardware specifications demanded for performance.

## 4.7 Test Cases & Validation

Due to the agile and iterative development process utilized during production, testing and validation were central elements to the project. To make the process easier, an excel spreadsheet was written at the beginning of the project and utilized to keep track of new, current, and completed user stories, their steps for completion, actual and expected results, story point value, associated tests, as well as their related area of functionality. This process enabled the easy tracking of completion of certain functionalities, assisting in the understanding of which features were considered complete, areas in which error handling or validation needed to be applied further as well as enriching the development process and building a dynamic testing approach. An example of some of these test cases can be seen in figure 4.18.

## 4.8 Scalability and Future Improvements

For the purposes of this project, a smaller and simplistic model was selected for functionality and implementation. In later iterations it could prove beneficial to experiment with larger models with greater capabilities and less of a generalized knowledge base. This could potentially improve the observed accuracy of predictions and further enhance the capabilities of the DM entity within game-play. Furthermore, experimentation with a variety of situations and test cases could greatly assist in this enhancement or development.

Further expansions include the potential of broadening the AI functionalities to assist during the online aspects of the app. A simple idea for a model which could be used within the system and immediately prove beneficial for players would be an "AI Assistant" in which the new model observes the current state of the game through the CombatManager.cs and CombatLogger.cs objects combat_log.json. Training using a variety of scenarios and leveraging the models natural language generation and inference abilities could construct a model that responds to context specific user queries to construct an appropriate response to aid their situation. This was initially considered during the conception of the project but was abandoned early in development due to time constraints, this will be addressed further in chapter 5.

# Chapter 5

# System Evaluation

The following chapter will cover an evaluation of the system in relation the objectives discussed during the introduction 1. It will highlight the strengths and weaknesses of the system and present the results of testing and critically reflecting on the design decisions made as well as areas for future improvement.

## 5.1 Evaluation of Project Objectives

The project was set out to create a networked, AI driven and readily accessible application which adapts the core concepts of Dungeon & Dragons into an interactive experience. The following table outlines how each of the overall objectives of the game have been met 5.1.

This table highlights how the project has succeeded in meeting the objectives set during the introduction and design phase. The general flow of the system meets the expectations set out during conception, with players being able to freely create, alter, and even use their own characters immediately upon starting up the application. The campaign related functionalities within the application exceeded the expectations as it evolved from a simplistic idea where players would simply design a static page, into a system in which there exists an interactive map and scene designing feature where unique scenarios may be constructed, then played. In addition, the character and campaign functionalities were well integrated together, mostly due to the design of the management objects utilized such as the main CampaignManager, alongside the GameplayManager and CombatManager. Within these systems player characters can effectively be transferred over the network, then freely interacted with during live game-play.

The modular, isolated and abstracted design of the system and its various components proved to be incredibly beneficial as components and their related functionalities could be altered or extended during development with little worry

of whether changes would disable or disrupt other core parts of the system. Furthermore, due to Unity's editing environment, the process of abstraction relating to classes used within the code behind was made extremely easy, as components can still be accessed within the backend no matter which directory they are defined in. Further design decisions relating to scripting and managing the backend services also proved beneficial. These include the application of advanced software design principles such as singletons, which were extensively used to create a manageable and stable system.

Evaluation of the various networking functionalities has proven that their performance also meets expectation, the metrics of which will be discussed in this projects sister paper. Overall, the functionalities related to ensuring efficient and consistent state management within the system performed as expected, with any changes made to game state such as attack and player health consistency along with player token placement being kept consistent throughout the application during execution.

## 5.2 AI Evaluation

Alongside the broad project objectives, multiple AI driven objectives were established during the projects conception. The vast majority of which have been achieved and demonstrated effectively within the project. However, within the set objectives exist a handful that retrospectively were either ambitious, implausible, or failed to meet due to circumstances and time. A full comparison of these results can be seen in 5.2.

In general, the main objectives established related to AI functionalities both within the project and during the research phase have been achieved. An understanding of the basic components and applications of AI and advanced learning algorithms within modern applications such as streaming platforms as well as more historic and static applications such as video games has been constructed. During the research, a satisfactory understanding of the processes, behaviors, and decision making capabilities of these models has been achieved.

Further understanding of the theoretical and modern proposed applications of advanced LLM's within static environments has also been achieved and displayed, with propositions within research for assistants as well as game-play or experience modifiers has also been highlighted within the proposed design and implemented system.

### 5.2.1 Advantages & Disadvantages

**Advantages**

The primary advantage of the implemented AI system is that it achieves its core design intent: the creation of an enemy that genuinely adapts to the player's behavior rather than following a predetermined script. The Dungeon Master boss fight presents a novel game-play experience in which a player's own choices such as their class, their spells, and their combat style directly inform the difficulty they face. This successfully demonstrates in a practical context how LLM-based inference can be applied as a game-play mechanic rather than purely as a generative or assistive tool, bridging a gap between the theoretical propositions encountered during the research phase 3 and a working implementation.

The system also maintains a deliberately low resource footprint. The fully trained model occupies approximately 400-900MB on disk and operates entirely on CPU without requiring a dedicated GPU or an internet connection. This satisfies the design requirement of keeping the AI self-contained and device-local, making it accessible to a broader range of hardware than a GPU-dependent solution would allow. The pipeline architecture in which `AIManager`, `DMBrain`, and `DMEnemyController` operate as discrete, loosely coupled components also means the AI subsystem can be extended or replaced independently of the rest of the application.

Furthermore, the project has established a reusable blueprint for how domain-specific LLM fine-tuning, on-device inference via LLamaSharp, and structured JSON communication can be combined within a Unity project. This has practical value beyond the immediate application, as the same pattern could be applied to other adaptive AI scenarios in future work.

**Disadvantages**

The most significant practical disadvantage is the inference latency. Running a language model on CPU introduces a noticeable delay between the end of the player's turn and the AI's response, which is somewhat acceptable in a turn-based game but would not be viable in a real-time context. During testing, it was found that some inference windows would range between 1-5 minutes, a delay that is understandably unacceptable for most game systems. An alternative implementation was identified during development in which the model is served locally via an Ollama endpoint, with `DMBrain` making HTTP requests to a `localhost` port rather than running inference in-process. This approach would yield significantly faster response times and prompt less of a computational burden; however, it was not adopted as it would require the player to have Ollama installed and the model pre-loaded separately from the application, violating the design requirement of a

fully self-contained, zero-dependency AI system.

The 768-token context window imposes a hard limit on how much observation history can be passed to the model in a single call. As the number of turns or the variety of actions grows, there is a risk that the observation JSON approaches or exceeds this limit, which would cause inference to degrade or fail. In practice, this was periodically encountered during test fights against the DM, with accuracy varying depending on the number of actions taken and the length of the fight, this represents a concern for the integrity of the model and its ability to process larger encounters.

Memory issues on lower-end hardware also produced occasional freezes during inference, as `DMBrain` executes model inference on a background thread, but the Unity main thread must wait for the result before advancing the turn. On machines with limited RAM or slow storage, this greatly interrupted the game-play. Collectively, these disadvantages reflect the current maturity ceiling of small on-device LLMs for real-time applications and point toward improvements that may be applied in the future.

| **Objective** | **Status** | **Comments** |
|---|---|---|
| Application of Project Management techniques | Achieved | Usage of users stories, frequent communication and weekly meetings are documented and greatly assisted in the development process |
| Adapt Character Creation | Achieved | Players are able to freely create and alter characters with real information from D&D 5e |
| Adapt Campaign Creation | Achieved | Campaigns can be freely created, managed and saved with basic information |
| Implement Scene making behaviors | Achieved | Within a campaign, players can create scenes with a variety of map making tools and tiles |
| Enable effective multiplayer session creation | Achieved | DM's can create a session for a campaign and distribute a code to allow for players to join |
| Ensure consistent state within campaigns during connections | Achieved | `PlayerConnectionManager` and `CombatManager` ensure the state of the game is distributed consistently |
| Implement AI learning functionalities | Achieved | Player can now engage with an entirely AI driven boss fight |
| Application of Software Design Principles and best practices | Achieved | Various components and management scripts were designed and implemented with these principles in mind |

Table 5.1: Overall Project Objectives evaluation

| **Objective** | **Status** | **Comments** |
|---|---|---|
| Investigate the application of local LLMs within a real-time game context | Achieved | Solid understanding of theoretical applications of LLMs within games was achieved during research |
| Research the history of "AI" within video games | Achieved | Understanding of core historical concepts such as Finite State Machines within games was found during research |
| Evaluate suitable AI models & data sets for on-device game integration | Achieved | Utilization of resources such as HuggingFace provided a deep understanding of a variety of models and data sets |
| Design and train a domain-specific model for player behaviour classification | Achieved | Usage of resources such as python training scripts and Google Colab enabled the fluid training and downloading of a domain specific model |
| Implement an Adaptive AI within Unity to enhance the player experience | Achieved | Implementation of DMBrain.cs, DMEnemyController.cs, AIManager alongside LlamaSharp enables the proposed system |
| Implement an AI driven assistant to help players during online sessions | Not Achieved | Due to time and resource restraints as well as a situational setback, this application had to be scrapped |
| Ensure AI functionalities are Isolated and non-reliant on API's or external services | Achieved | The AI operates entirely within the user's device without the aid of external resources. |

Table 5.2: Evaluation of the AI Objectives established

# Chapter 6

# Conclusion

## 6.1 Summary

"Dungeons Together" set out to build an interactive, readily accessible, multiplayer game which adapts the core functionalities of Dungeons and Dragons whilst also offering unique developments of an innovative AI driven boss fight. As covered in chapter 5 the main objectives related to the core design and development of the application were achieved, leading to a project that not only adapts a dynamic and extensive table-top system, but also applies principles and best practices in software design relating to software design and program development. Ultimately, this led to the creation of an app with a modular and abstracted structure in which new functionalities may be freely added, developed and applied directly to the system in the future without fear of disrupting other functionalities.

Further objectives within the application include the development and addition of a unique networking functionality to demonstrate an understanding of Peer-2-Peer systems as well as solutions to the problems they produce, but also to ensure an effective online multiplayer functionality in which all clients connected to the dungeon master are kept up to date on all recent changes in state. Additionally, the project sought to adapt LLM technologies in the form of an advanced AI functionality in which the user's own actions directly inform the decision making process of an enemy character. This was ultimately achieved through the use of the Qwen2.5-0.5-Instruct model, which was trained using context and application specific data. This ultimately sought to create a unique experience that not only demonstrates the capabilities and potential roles that AI and LLM can play within the development of games beyond the generation of plain assets or text observed within other systems, but also demonstrates and confirms a practical application of AI within games based on the theories covered in chapter 3.

Although the AI model and system still show room for improvement, a sound

and reusable process relating to model selection, training, and implementation has been established within this project, allowing for replication of the process for future models ultimately for the enhancement of these systems and vidoe games in general through the application of further training or enhanced response restriction as covered in 5.

## 6.2 Key Achievements

### 6.2.1 Deepened Understanding of AI systems

One of the greatest achievements observed throughout development was the acquisition of a broader and deeper understanding of the operation of AI and advanced learning algorithms within modern applications. There exists a concrete understanding of the many uses for these systems, such as those outlined during the research performed in chapter 3, most prominently recommendation algorithms utilized by applications such as YouTube and Spotify. Such applications demonstrated adaptive learning capabilities based on a broad range of areas, ultimately inspiring the system implemented within this project.

In addition, a better understanding of the algorithms that drive behavior systems in video games in the past has also been established. Systems such as the previously mentioned Finite-State Machines and Behavioral Trees utilized to configure different aspects of behavior for NPC's and enable the development and display of seemingly autonomous behavior, now commonly referred to as simply "AI".

### 6.2.2 AI Design & Training Process

Arguably the greatest achievement obtained throughout the development process is the understanding, application, and ability to design a specific context aware LLM and then apply this model to a static system like a video game. The process of understanding the differences and trade-offs offered by the large variety of models available today, such as context window, inference speed and memory consumption, then ultimately selecting a model demonstrates an advanced understanding and research contribution.

Furthermore, the derivation of training data directly from the game's own ability and combat systems to train the model based on grounded contextual information to reduce hallucinations, risks and improve the model's classification process has ultimately created a scalable and reusable process in which future models may be trained, new data may be inferred and increasingly advanced models may be applied to this system along with others.

### 6.2.3 Software Design Principle Application

A more immediate practical achievement observed within the system design is the aforementioned application of software design principles. The practical application of practices such as abstraction, isolation, and further design choices such as the usage of singletons to handle access and further managing state assisted in cementing these principles beyond merely theoretical knowledge into a demonstration of how they may be applied to truly enhance all software systems where applicable.

## 6.3 Limitations Encountered

As discussed in the previous chapter 5, the main limitations faced within the practical aspects of the application were related to the accuracy of the model, as well as hallucinations and errors. Given more time, it is almost guaranteed that these systems would be able to act more efficiently and with higher accuracy of predictions and responses.

Other limitations faced during development related to personal issues encountered which impeded the ability for code to be written or models to be trained for a period of time towards the beginning, middle, and end of the project's life cycle.

## 6.4 Final Remarks

Overall, the Dungeons Together development process has proved to be an incredibly informative experience, in which new technologies were experimented with, and theoretical knowledge was transformed into practical experience. The Agile and iterative development process utilized throughout the entire project proved to be incredibly beneficial, as frequent communication, iterative development, and testing meant that errors were caught early and changes in design could be made without disrupting or destroying previous work. The project demonstrates an application which applies networking technologies, adaptive LLM aspects as well as design principles to create an enjoyable, interactive, and memorable environment.

Dungeons & Dragons represents a creative and collaborative table top experience that countless people around the world enjoy. The development of this project has proven to be an enriching experience, acquiring knowledge of developing, training, and deploying artificial intelligence in areas of personal interest. As well as this, the process of developing and adapting a game based on a personal topic made the entire process feel immensely more engaging, Dungeons Together captures some of the magic from the original game, and encapsulates it within a new interactive application. Dungeons & Dragons and subsequently Dungeons Together have always been about people, experiencing and creating, together.

# Bibliography


[1] Akhilesh Kumar Sharma, Shamik Tiwari, Gaurav Aggarwal, Nitika Goenka, Anil Kumar, Prasun Chakrabarti, Tulika Chakrabarti, Radomir Gono, Zbigniew Leonowicz, and Michał Jasiński. Dermatologist-level classification of skin cancer using cascaded ensembling of convolutional neural network and handcrafted features based deep neural network. *IEEE Access*, 10:17920–17932, 2022.

[2] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, RecSys '16, page 191–198, New York, NY, USA, 2016. Association for Computing Machinery.

[3] Carlos A. Gomez-Uribe and Neil Hunt. The netflix recommender system: Algorithms, business value, and innovation. *ACM Trans. Manage. Inf. Syst.*, 6(4), December 2016.

[4] James Davidson, Benjamin Liebald, Junning Liu, Palash Nandy, Taylor Van Vleet, Ullas Gargi, Sujoy Gupta, Yu He, Mike Lambert, Blake Livingston, and Dasarathi Sampath. The youtube video recommendation system. In *Proceedings of the Fourth ACM Conference on Recommender Systems*, RecSys '10, page 293–296, New York, NY, USA, 2010. Association for Computing Machinery.

[5] Youtube's ai is the puppet master over most of what you watch. `https://www.cnet.com/tech/services-and-software/youtube-ces-2018-neal-mohan/`. Accessed: 2026-03-29.

[6] Dr. Geetanjali Kale Puja Deshmukh*. International Journal of Scientific Research in Computer Science, Engineering and Information Technology. 2018.

[7] Julian Togelius Georgios N. Yannakakis. *Artificial Intelligence and Games*. Springer, 2018-01-26.

[8] Roberto Gallotta, Graham Todd, Marvin Zammit, Sam Earle, Antonios Liapis, Julian Togelius, and Georgios N. Yannakakis. Large language models and games: A survey and roadmap. *IEEE Transactions on Games*, pages 1–18, 2024.

[9] Eugene Martynenko. The pros and cons of unity game engine - https://pinglestudio.com/blog/full-cycle-development/pros-and-cons-of-unity-game-engine, 2025.

[10] Unity. Unity main documentation - https://docs.unity.com/en-us, 2025.

[11] Video Game Insights. The big game engine report, 2025.

[12] Official Unity 2D Game Development Documentation - https://docs.unity3d.com/Manual/Unity2D.html.

[13] Penny Sweetser. Large language models and video games: A preliminary scoping review. In *Proceedings of the 6th ACM Conference on Conversational User Interfaces*, CUI '24, New York, NY, USA, 2024. Association for Computing Machinery.

[14] Official Ollama Documentation - https://github.com/awaescher/OllamaSharp.

[15] Official NuGet Documentation - https://github.com/GlitchEnzo/NuGetForUnity.

[16] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: efficient finetuning of quantized llms. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, 2023. Curran Associates Inc.

[17] Yousef Fazea, Zainab S. Attarbash, Fathey Mohammed, and Ibrahim Abdullahi. Review on unstructured peer-to-peer overlay network applications. In *2021 International Conference of Technology, Science and Administration (ICTSA)*, pages 1–7, 2021.

[18] Unity Netcode Documentation - https://docs.unity3d.com/Packages/com.unity.netcode.gameobjec
started-with-ngo.html.

[19] Unity relay documentation - https://docs.unity.com/en-us/relay. Unity Relay.

[20] Unity transport documentation - https://docs.unity3d.com/packages/com.unity.transport@6.5/ma
Transport Docs.

# Appendix A

# App Repo & Links

Below are some useful links related to the project for ease of access to those who are interested

- To see the complete code base and to get a better understanding of it's structure and cloning, please visit the GitHub Repository here
- Similarly, you can access the fully published game on our Itch.io page